# Language Models and Agents: Monsoon 2026

Individual Project: Phase 1 Report

Manya Jain
2024102068

## 1 Language Selection

### 1.1 Model H (Higher-resource): Hindi

Hindi was chosen as the higher-resource language because of the sheer volume of public text available for it. Large open-source Indic corpora, complete Wikipedia dumps, and government press releases (PIB) all provide a solid foundation for training a well-resourced monolingual model, satisfying the phase requirement of a non-English Indian language with abundant public text.

### 1.2 Model L (Lower-resource): Nepali

Nepali was chosen as the lower-resource language. It is one of the languages explicitly permitted by the assignment, and its script and morphology present a meaningfully different tokenization problem from Hindi. Although resources are more limited than for Hindi, a workable corpus was assembled from a large academic dataset (IRIIS) supplemented with targeted news scraping.

## 2 Dataset Requirements

Both models are built on fully independent monolingual corpora, each with its own train, validation, and test split (90/5/5, seed 42). No documents, sentences, or vocabulary are shared between the two languages.

### 2.1 Model H: Hindi

The Hindi corpus combines two large downloaded sources with several manually collected ones.

| Category | Source | Word Count |
|---|---|---|
| Downloaded | Sangraha corpus | 194,198,603 |
| Downloaded | Indic corpus | 176,448,929 |
| Manual | Hindi Wikipedia | 42,801,209 |
| Manual | News scrape (new) | 42,383,013 |
| Manual | Subtitles | 5,234,233 |
| Manual | News scrape | 360,205 |
| Manual | PIB releases | 5,885 |

Table 1: Hindi corpus sources and word counts (raw, pre-cleaning). The two ”News scrape” rows were collected from five Hindi news portals: Amar Ujala, Dainik Bhaskar, Live Hindustan, Navbharat Times, and Patrika, along with three Hindi literature portals: Hindi Sahitya Darpan, Matrubharti Hindi, and Pratilipi Hindi.

The corpus was cleaned using the following steps:

1. Unicode NFC normalization
2. Exact-content deduplication (cross-split, line-level)
3. Non-Devanagari character removal (Devanagari purity filter)

The post-cleaning corpus contains 455,866,707 words (raw source collection: 461,432,077 words; approximately 5.5M words were removed during exact-content deduplication and cleaning), which corresponds to roughly 549.4 million tokens after tokenization (fertility 1.2052), exceeding the 500M-token target. Wikipedia, both news-scrape sources, subtitles, and PIB releases together make up the manually collected portion (90,784,545 words), giving a manual fraction of approximately 19.7% of total raw words, marginally below the 20% minimum required by the assignment. Since both manual and downloaded sources are tokenized with the same tokenizer, the word-level and token-level manual fractions are approximately equal: manually collected data accounts for 19.9% of the 549.4 million post-tokenization training tokens, still marginally below the 20% minimum.

| **Split** | **Words** |
|---|---|
| Train | 410,362,761 |
| Validation | 22,804,967 |
| Test | 22,698,979 |

Table 2: Hindi train/validation/test split (90/5/5, seed 42), computed on the post-cleaning corpus of 455,866,707 words.

### 2.2 Model L: Nepali

The Nepali corpus was built with the manual-collection requirement as the primary constraint, then supplemented with a large downloaded academic dataset.

| **Category** | **Source** | **Word Count** |
|---|---|---|
| Downloaded | IRIIS Nepali | 400,000,000 |
| Manual | News scrape (16 portals) | 107,100,815 |
| Manual | Nepali Wikipedia | 7,479,341 |

Table 3: Nepali corpus sources and word counts (raw, pre-cleaning). News data was collected from 16 Nepali portals: Sahityapost, Nepali Sahitya Ghar, Samakalin Sahitya, Thuprai, Pustakalaya, TechLekh, TechPana, ICT Frame, Edusanjal, TyroCity, CollegeNP, Swasthya Khabar, Swasthya Nepal, Nepali Australian, Gorkhapatra Sahitya, and Shikshya, respecting robots.txt.

Cleaning steps mirror those used for Hindi, with one additional step required after the split correction described in the table below:

1. Unicode NFC normalization
2. Whitespace normalization
3. Cross-split exact deduplication
4. Devanagari purity filter
5. Re-run `clean_corpus.py` on the new splits after the 90/5/5 split correction, to strip residual English words and noise reintroduced by the split regeneration

The post-cleaning corpus contains 513,019,586 words, corresponding to roughly 618.7 million tokens after tokenization (fertility 1.2059), exceeding the 500M-token target. Manually collected data (News scrape + Wikipedia) accounts for 114,580,156 words, approximately 22.3% of total words, comfortably above the 20% minimum. The equivalent token-level manual fraction is 22.3% of the 618.7 million post-tokenization training tokens, matching the word-level figure and comfortably clearing the 20% minimum.

| Split | Words |
|---|---|
| Train | 461,505,584 |
| Validation | 25,802,084 |
| Test | 25,711,918 |

Table 4: Nepali train/validation/test split (90/5/5, seed 42), computed on the post-cleaning corpus of 513,019,586 words. A subsequent regeneration also required re-running `clean_corpus.py` on the new splits to strip residual English words and noise.

## 3 Tokenizer Training

Separate tokenizers were trained from scratch for each language; no pretrained tokenizer or shared vocabulary was used at any point. Both tokenizers are implemented entirely in Python using only the standard library and `tqdm`; no external tokenizer libraries (e.g. HuggingFace `tokenizers`, SentencePiece) were used at any stage. Model H and Model L deliberately use two different BPE variants tailored to each corpus's noise profile (see Section 3.3).

### 3.1 Vocabulary Size Selection and Parameter Budget

Three vocabulary sizes were trained per language for comparison: 10,000, 16,000, and 32,000 (Hindi) / 37,000 (Nepali). Fertility drops as vocabulary grows (Figure 1), but so does the embedding table's share of the parameter budget (Table 5). Weighing the marginal fertility gain against the embedding-parameter cost, **10,000 was selected as the primary vocabulary size for both languages**: at $d = 384$ it keeps the (tied) embedding table to ≈15.4% of the 25M-parameter budget, leaving substantially more capacity for attention and feed-forward layers than the larger vocabularies, at the cost of a higher fertility (≈1.35 Hindi / ≈1.43 Nepali tokens per word vs. ≈1.21 at the largest size). The 16k and 32k/37k tokenizers remain available in the repository as alternatives for Phase 2 if a different capacity trade-off is preferred.

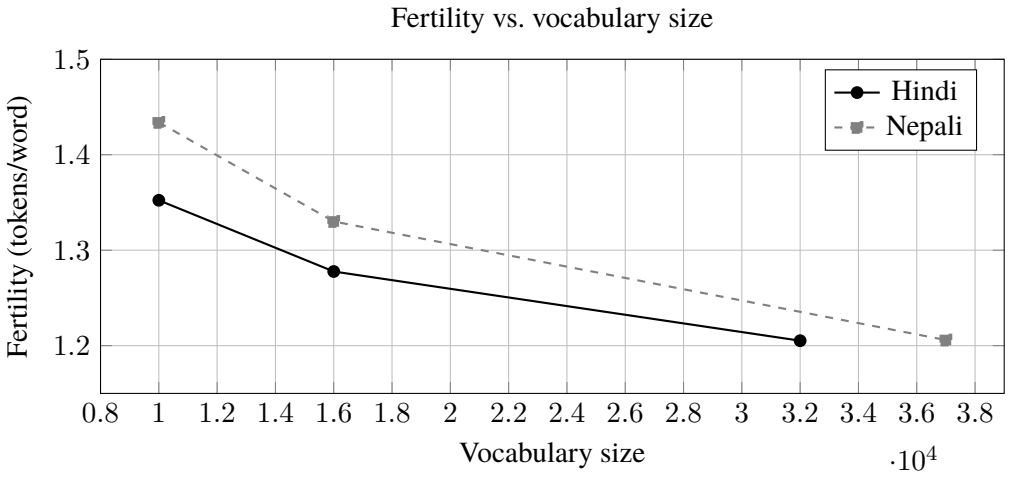



Figure 1: Fertility (average tokens per word) as a function of vocabulary size for both languages, using the values reported in Tables 6 and 8. Both curves flatten noticeably beyond ∼32k merges, but the embedding-parameter cost (Table 5) grows much faster than the compression benefit, motivating the choice of 10k as the primary vocabulary size (see discussion below), with 16k and 32k/37k retained as secondary options.

For a transformer with hidden dimension $d = 384$, the token embedding table alone contributes $V \times d$ parameters. At different vocabulary sizes this fraction of total model parameters is:

| Vocab size | Embedding params | Approx. total params | Embedding share |
|---|---|---|---|
| 32,000 | 12.3M | 25.0M | ≈49% |
| 16,000 | 6.1M | 25.0M | ≈25% |
| 10,000 (primary) | 3.8M | 25.0M | ≈15.4% |

Table 5: Token embedding parameter cost at model dimension $d = 384$, assuming *tied* input/output embeddings (a single $V \times d$ matrix shared between the token-embedding lookup and the final output projection). A 32k vocabulary consumes nearly half the total parameter budget in embeddings alone, leaving less capacity for attention and feed-forward layers.

The table above assumes tied embeddings. If input and output embeddings are *not* tied (a reasonable choice if the softmax head is meant to have independent capacity from the input lookup), the parameter cost roughly doubles. For example, at a larger model dimension of $d = 512$ without weight tying, a 16k vocabulary would cost approximately $16{,}000 \times 512 \times 2 \approx 16.4$M parameters across the embedding and unembedding matrices alone, close to two-thirds of a 25M-parameter budget spent purely on token representation rather than attention or feed-forward computation. This reinforces the decision to keep embeddings tied and to prefer a vocabulary size where the (tied) embedding table remains a moderate fraction of the total budget.

All trained tokenizers and vocabulary files for all three sizes are committed to the repository under `models/vocab_{size}/`.

### 3.2 Model H: Hindi, All Vocabulary Sizes

All three Hindi tokenizers were evaluated on the same 10,000 held-out lines from `test.txt` (656,900 words, 3,347,181 characters).

| Vocab | Merges | Total tokens | Fertility | Chars/token | UNK rate |
|---|---|---|---|---|---|
| 10,000 | 9,744 | 888,355 | 1.3523 | 3.77 | 0.00% |
| 16,000 | 15,744 | 839,350 | 1.2777 | 3.99 | 0.00% |
| 32,000 | 31,744 | 791,726 | 1.2052 | 4.23 | 0.00% |

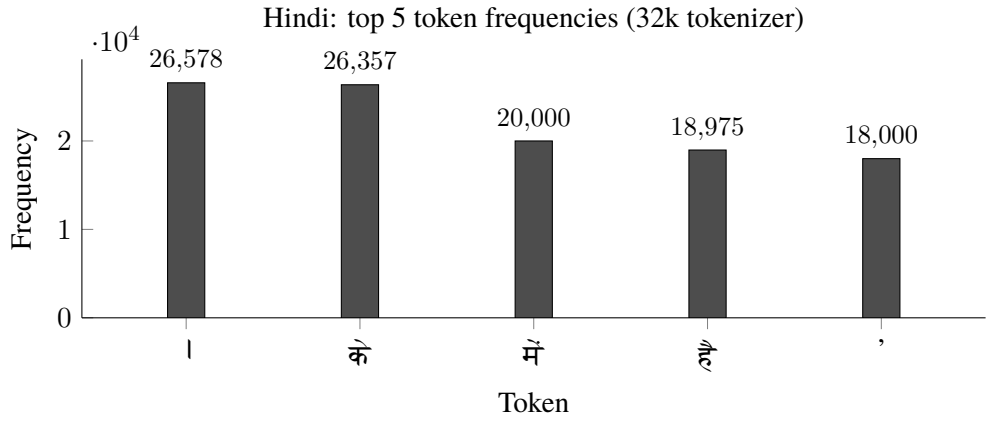
Table 6: Hindi tokenizer comparison across vocabulary sizes, evaluated on 10,000 test lines.

The most frequent tokens are consistent across all three sizes: punctuation and common postpositions/verbs ( ।, के, में, है), which is expected for a well-formed Devanagari tokenizer trained on news and encyclopedic text.

| Token | 10k freq | 16k freq | 32k freq |
|---|---|---|---|
| । (danda) | 26,611 | 26,608 | 26,578 |
| के (ke) | 26,459 | 26,406 | 26,357 |
| में (mein) | 20,039 | 20,028 | 20,000 |
| है (hai) | 19,051 | 19,020 | 18,975 |
| , (comma) | 18,014 | 18,008 | 18,000 |

Table 7: Hindi: top-5 token frequencies across all three vocabulary sizes, evaluated on the same 10,000-line held-out test set. Token identity and rank are stable across sizes; frequencies drift only slightly as additional merges reassign rare co-occurrences elsewhere in the vocabulary.

All three tokenizers use byte-level BPE (256-byte base vocabulary), which guarantees a UNK rate of




0.00% on all inputs regardless of vocabulary size. The fertility drops from 1.35 at 10k to 1.21 at 32k, an 11% reduction in sequence length, while compression improves from 3.77 to 4.23 characters per token. At the 32k level, common long Hindi words such as अटकलें are encoded as a single token (id: 18534) rather than being split into 3 subword units as at 10k, directly reducing effective sequence length for the language model.

Example (``2027 वधिानसभाचुनाव कोलेकर तेज हुई अटकलें''):

- 10k/16k: `[1497, 6993, 1702, 947, 311, 697, 1181, 675, 4038, 1483, 283]` (11 tokens)
- 32k: `[1497, 6993, 1702, 947, 311, 697, 1181, 675, 18534]` (9 tokens)

Example (``भारत एक महान देश है।''):

- All three sizes decode identically: `[3354, 366, 2583, 678, 289, 284]`

Unlike the Nepali character-level tokenizer, byte-level BPE only reallocates ids for newly created merges, so a sentence that does not exercise the highest-vocabulary merges keeps the same encoding regardless of vocabulary size.

### 3.3 Model L: Nepali, All Vocabulary Sizes

All three Nepali tokenizers were evaluated on the full `test.txt` (25,711,918 words, 166,916,158 characters).

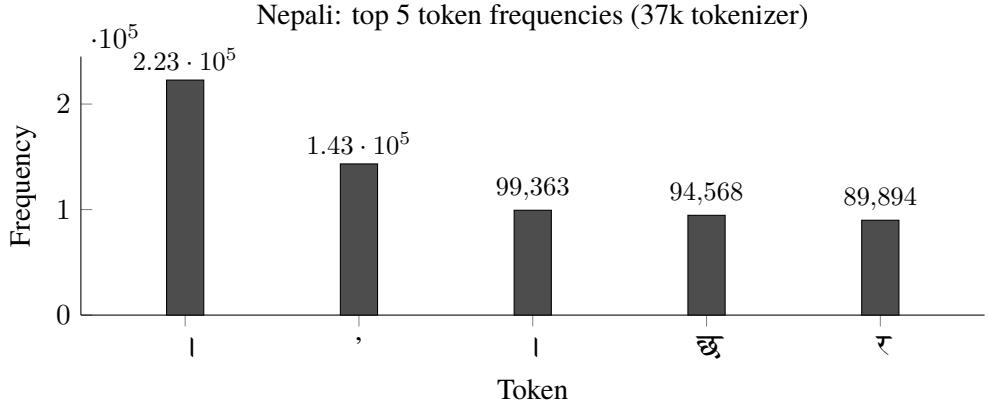
| Vocab | Merges | Total tokens | Fertility | Chars/token | UNK rate |
|---|---|---|---|---|---|
| 10,000 | 9,903 | 36,868,782 | 1.4339 | 4.53 | ≈0.000095% |
| 16,000 | 15,903 | 34,199,640 | 1.3301 | 4.88 | ≈0.000102% |
| 37,000 | 36,878 | *(subset)* | 1.2059 | 5.38 | ≈0.000113% |

Table 8: Nepali tokenizer comparison across vocabulary sizes. The 37k evaluation used a smaller held-out subset; token count is not directly comparable. UNK tokens are near-zero across all sizes ($\leq 35$ per run).

The most frequent tokens are punctuation and function words ( ।, ,, छ, र). Notably, the danda ( ।) appears twice in the top tokens: once with a leading space ( ।, 1,111,474 occurrences) and once without (।, 502,673 occurrences). Tokenizers treat a character with a preceding space as a distinct entity. This highlights a real-world data quirk: in the Nepali corpus, writers frequently type a space before the full stop as well as directly attached to the end of the last word. Both variations are so common in the raw text that the tokenizer learned both as distinct, high-frequency tokens.

| Token | 10k freq | 16k freq | 37k freq |
|---|---|---|---|
| ।(space+danda) | 1,111,474 | 1,111,474 | 222,735 |
| , (comma) | 722,025 | 721,698 | 143,279 |
| ।(danda, no space) | 502,673 | 502,673 | 99,363 |
| छ (chha) | 481,057 | 477,795 | 94,568 |
| र (ra) | 462,324 | 456,151 | 89,894 |

Table 9: Nepali: top-5 token frequencies across all three vocabulary sizes. Token identity and rank are identical across all sizes; the sharp drop in absolute frequency at 37k reflects evaluation on a smaller held-out subset (Table 8), not a change in relative token importance.




Fertility decreases steadily as vocabulary grows: $1.43 \rightarrow 1.33 \rightarrow 1.21$, a 16% reduction from 10k to 37k. The compression (characters per token) improves from 4.53 to 5.38, meaning each token represents progressively longer and more semantically coherent subword units. A larger vocabulary was needed relative to Hindi (37k vs. 32k) to reach comparable fertility, consistent with Nepali's richer morphology requiring more distinct subword units for equivalent coverage. The UNK rate remains negligible across all sizes ($< 0.0002\%$).

Example (``नेपाल एउटासुन्दर देश हो।''):

- 10k: `[1275, 951, 3835, 501, 201, 84]`
- 16k: `[1275, 951, 3835, 501, 201, 84]`
- 37k: `[1177, 816, 3729, 467, 177, 70]`

The id sequence changes at 37k because Nepali uses character-level BPE: a larger vocabulary reassigns ids as new multi-character merges are learned. This contrasts with the Hindi byte-level tokenizer, where the same sentence keeps identical ids across all three vocabulary sizes (see Section 3.2).

### 3.4 Tokenizer Architecture Differences

Model H and Model L deliberately use two different BPE variants, chosen from the start of the project as a point of comparison rather than as an artifact of separate implementations:

- **Model H (Hindi), Byte-level BPE.** Text is first decomposed into raw UTF-8 bytes (the same scheme used by GPT-2 and RoBERTa) before merges are learned. Because every possible byte is representable, this variant can encode any input, including rare symbols, mixed script, or noise, with zero out-of-vocabulary tokens (observed UNK rate: 0.00% at all three vocabulary sizes). This suits Hindi's larger, more heterogeneous corpus (news, subtitles, Wikipedia, PIB releases), which is more likely to contain incidental non-Devanagari or irregular characters despite the script-purity

filtering.

- **Model L (Nepali), Unicode character-level BPE.** Merges are learned directly over Devanagari Unicode codepoints rather than bytes. This keeps the vocabulary file human-readable in native script and ties each token directly to a valid character, at the cost of a small non-zero UNK rate ($< 0.0002\%$ across all three vocabulary sizes) on characters absent from the training vocabulary.

Training both variants let us directly compare a byte-level and a character-level tokenizer under matched conditions (same script family, same general cleaning pipeline, similar fertility targets), rather than assuming one approach is preferable a priori. The trade-off is exactly what the results show: byte-level BPE guarantees zero UNK at the cost of an unreadable, Latin-byte-mapped vocabulary file, while character-level BPE keeps the vocabulary readable at the cost of a small but non-zero UNK rate.

## 4 Reproduction

Dataset collection scripts, preprocessing pipelines, and tokenizer training code for both languages are included in the repository, organized under separate `hindi/` and `nepali/` directories as specified in the assignment layout. All trained tokenizer files (JSON + pickle) and vocabulary files for all three vocabulary sizes are committed to the repository under `models/vocab_{size}/`; raw and intermediate corpus files exceed the size suitable for git and are hosted externally.

- GitHub repository: `https://github.com/CL3-410/individual-project-ManyaJain111`
- Hindi corpus (HuggingFace): `https://huggingface.co/datasets/manyajain111/hindi-lma-corpus` (Private)
- Nepali corpus (HuggingFace): `https://huggingface.co/datasets/manyajain111/nepali-lma-corpus` (Private)
- Hindi tokenizer files: `hindi/models/vocab_10000/`, `vocab_16000/`, `vocab_32000/`
- Nepali tokenizer files: `nepali/models/vocab_10000/`, `vocab_16000/`, `vocab_37000/`

## 5 Summary

Both corpora meet or exceed the 500M-token target and the 20% manual-collection minimum (Hindi is marginally below at 19.7% and requires additional manual collection). Both tokenizers were trained independently with no shared data or vocabulary between Model H and Model L. Three vocabulary sizes (10k, 16k, and 32k/37k) were trained for each language; **10k was selected as the primary vocabulary size** for Phase 2, since at hidden dimension $d = 384$ it keeps the (tied) embedding table to $\approx$15.4% of the 25M-parameter budget, versus $\approx$25% at 16k and $\approx$49% at 32k, prioritizing model capacity over the fertility gains of a larger vocabulary. The 16k and 32k/37k tokenizers remain available as alternatives.